# LABSHEET-10

## Q1. Initialize a Git repository for the project, add a proper .gitignore, and commit the code in logical steps using Git commands.

```
mkdir PHP_Project
cd PHP_Project
git init
touch index.php .gitignore
```

### .gitignore

```
.env
/vendor/
*.log
git add .
git commit -m "Initial project setup"
git status
```

## Q2. Create a feature branch, make changes, and merge it into the main branch. Demonstrate how to resolve a merge conflict and push the final code to GitHub.

```
git branch feature-update
git checkout feature-update
```

Make changes in index.php, then:

```
git add .
git commit -m "Feature update"
git checkout main
git merge feature-update
```

Resolve any conflict in the affected file, then:

```
git add .
git commit -m "Resolve merge conflict"
git remote add origin YOUR_GITHUB_REPOSITORY_URL
git push -u origin main
```

## Q3. Create a basic MVC folder structure for a PHP project.

```
mkdir -p MVC_Project/{config,controllers,models,views,public}
touch MVC_Project/public/index.php
touch MVC_Project/config/database.php
touch MVC_Project/controllers/HomeController.php
touch MVC_Project/models/User.php
touch MVC_Project/views/home.php
```

## Q4. Implement a Front Controller pattern using a single index.php file to route requests to different controllers.

### public/index.php

```php
<?php
require_once '../controllers/HomeController.php';

$page = $_GET['page'] ?? 'home';

switch ($page) {
    case 'home':
        $controller = new HomeController();
        $controller->index();
        break;

    default:
        http_response_code(404);
        echo "404 Page Not Found";
}
?>
```

## Q5. Create a Database class using PDO Singleton Pattern and implement basic CRUD operations.

### Database.php

```php
<?php
class Database {
    private static $instance = null;
    private $conn;

    private function __construct() {
        $this->conn = new PDO(
            "mysql:host=localhost;dbname=testdb",
            "root",
            ""
        );
        $this->conn->setAttribute(
            PDO::ATTR_ERRMODE,
            PDO::ERRMODE_EXCEPTION
        );
    }

    public static function getInstance() {
        if (self::$instance == null) {
            self::$instance = new Database();
        }
        return self::$instance;
    }

    public function getConnection() {
        return $this->conn;
    }
}
?>
```

## Q6. Create reusable header, footer, and navigation templates using PHP include and require statements.

### header.php

```
<!DOCTYPE html>
<html>
<head>
   <title>PHP Project</title>
</head>
<body>
```

### nav.php

```
<nav>
   <a href="index.php">Home</a>
   <a href="about.php">About</a>
</nav>
```

### footer.php

```
<footer>
   <p>Copyright 2026</p>
</footer>
</body>
</html>
```

### index.php

```
<?php
include 'header.php';
include 'nav.php';
echo "<h1>Welcome to PHP Project</h1>";
include 'footer.php';
?>
```

## Q7. Develop a REST API in PHP that supports GET, POST, PUT, and DELETE operations and returns JSON responses.

```
<?php
header("Content-Type: application/json");

$method = $_SERVER['REQUEST_METHOD'];

switch ($method) {
   case 'GET':
      echo json_encode(["message" => "GET request"]);
      break;

   case 'POST':
      echo json_encode(["message" => "POST request"]);
      break;

   case 'PUT':
      echo json_encode(["message" => "PUT request"]);
      break;

   case 'DELETE':
      echo json_encode(["message" => "DELETE request"]);
      break;

   default:
```

```
    http_response_code(405);
    echo json_encode(["error" => "Method not allowed"]);
}
?>
```

## Q8. Implement token-based authentication in PHP and access a protected API using JavaScript Fetch API.

### api.php

```
<?php
header("Content-Type: application/json");

$headers = getallheaders();
$token = $headers['Authorization'] ?? '';

if ($token !== 'Bearer ' . getenv('API_TOKEN')) {
   http_response_code(401);
   echo json_encode(["error" => "Unauthorized"]);
   exit;
}

echo json_encode(["message" => "Access granted"]);
?>
```

### script.js

```
fetch("api.php", {
   method: "GET",
   headers: {
      "Authorization": "Bearer YOUR_API_TOKEN"
   }
})
.then(response => response.json())
.then(data => console.log(data));
```

## Q9. Implement CSRF protection in a PHP form using session-based CSRF tokens.

```
<?php
session_start();

if (empty($_SESSION['csrf_token'])) {
   $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
   if (!hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'] ?? '')) {
      die("Invalid CSRF Token");
   }
   echo "Form submitted successfully";
}
?>

<form method="POST">
   <input type="hidden" name="csrf_token"
        value="<?php echo htmlspecialchars($_SESSION['csrf_token']); ?>">
```

```
    <input type="text" name="username">
    <button type="submit">Submit</button>
</form>
```

## Q10. Configure secure PHP sessions using appropriate session cookie settings.

```
<?php
session_set_cookie_params([
    'lifetime' => 0,
    'path' => '/',
    'secure' => true,
    'httponly' => true,
    'samesite' => 'Strict'
]);

session_start();

session_regenerate_id(true);

$_SESSION['username'] = 'Suhani';

echo "Secure session started";
?>
```

## Q11. Validate and sanitize user input using PHP filter functions.

```
<?php
$email = filter_input(INPUT_POST, 'email', FILTER_VALIDATE_EMAIL);
$age = filter_input(INPUT_POST, 'age', FILTER_VALIDATE_INT);
$url = filter_input(INPUT_POST, 'url', FILTER_VALIDATE_URL);

if ($email && $age && $url) {
    echo "Valid Input";
} else {
    echo "Invalid Input";
}
?>
```

## Q12. Implement login attempt limiting in PHP to prevent brute-force attacks.

```
<?php
session_start();

if (!isset($_SESSION['attempts'])) {
    $_SESSION['attempts'] = 0;
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if ($_SESSION['attempts'] >= 5) {
        die("Too many attempts. Try again later.");
    }

    $username = $_POST['username'] ?? '';
    $password = $_POST['password'] ?? '';

    if ($username === 'admin' && $password === 'admin123') {
        $_SESSION['attempts'] = 0;
        echo "Login Successful";
```

```
    } else {
        $_SESSION['attempts']++;
        echo "Invalid Login";
    }
}
?>

<form method="POST">
    <input type="text" name="username">
    <input type="password" name="password">
    <button type="submit">Login</button>
</form>
```

## Q13. Create a Forgot Password system using secure token generation and password reset functionality.

```
<?php
$token = bin2hex(random_bytes(32));
$hashedToken = password_hash($token, PASSWORD_DEFAULT);

$expiry = date('Y-m-d H:i:s', time() + 3600);

echo "Reset token: " . $token;
echo "<br>Expiry: " . $expiry;
?>
```

## Q14. Configure PHPMailer to send emails using SMTP.

```
composer require phpmailer/phpmailer
```

### sendmail.php

```
<?php
use PHPMailer\PHPMailer\PHPMailer;
use PHPMailer\PHPMailer\Exception;

require 'vendor/autoload.php';

$mail = new PHPMailer(true);

try {
    $mail->isSMTP();
    $mail->Host = 'smtp.gmail.com';
    $mail->SMTPAuth = true;
    $mail->Username = getenv('SMTP_USERNAME');
    $mail->Password = getenv('SMTP_PASSWORD');
    $mail->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS;
    $mail->Port = 587;

    $mail->setFrom(getenv('SMTP_USERNAME'), 'PHP Project');
    $mail->addAddress('receiver@example.com');

    $mail->Subject = 'Test Email';
    $mail->Body = 'Email sent successfully';

    $mail->send();
    echo "Email sent successfully";
} catch (Exception $e) {
```

```
    echo "Email could not be sent";
}
?>
```

## Q15. Implement secure file uploading in PHP with file type and size validation.

```
<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $file = $_FILES['file'];

    if ($file['error'] === UPLOAD_ERR_OK && $file['size'] <= 2 * 1024 * 1024) {
        $finfo = new finfo(FILEINFO_MIME_TYPE);
        $mime = $finfo->file($file['tmp_name']);

        $allowed = ['image/jpeg', 'image/png'];

        if (in_array($mime, $allowed, true)) {
            $name = bin2hex(random_bytes(16)) . '.jpg';
            move_uploaded_file($file['tmp_name'], 'uploads/' . $name);
            echo "File uploaded successfully";
        } else {
            echo "Invalid file type";
        }
    } else {
        echo "Invalid file or file too large";
    }
}
?>

<form method="POST" enctype="multipart/form-data">
    <input type="file" name="file">
    <button type="submit">Upload</button>
</form>
```

## Q16. Create custom 404 and 500 error pages in PHP and implement error logging.

```
<?php
ini_set('display_errors', '0');
ini_set('log_errors', '1');
ini_set('error_log', __DIR__ . '/errors.log');

http_response_code(404);
include '404.php';
?>
```

### 404.php

```
<h1>404 - Page Not Found</h1>
<p>The requested page does not exist.</p>
```

### 500.php

```
<h1>500 - Internal Server Error</h1>
<p>Something went wrong.</p>
```

## Q17. Configure Apache .htaccess for clean URLs, HTTPS redirection, and disabling directory listing.

### .htaccess

```
Options -Indexes

RewriteEngine On

RewriteCond %{HTTPS} off
RewriteRule ^ https://%{HTTP_HOST}%{REQUEST_URI} [L,R=301]

RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^(.*)$ index.php?url=$1 [QSA,L]
```

## Q18. Configure a local self-signed SSL certificate for Apache.

```
sudo apt update
sudo apt install apache2 openssl
sudo a2enmod ssl
sudo mkdir -p /etc/apache2/ssl
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/apache2/ssl/server.key \
-out /etc/apache2/ssl/server.crt
sudo systemctl restart apache2
```

## Q19. Create an Apache Virtual Host for a custom local domain and map it using the hosts file.

```
sudo mkdir -p /var/www/myproject/public
sudo nano /etc/apache2/sites-available/myproject.conf
```

### myproject.conf

```
<VirtualHost *:80>
   ServerName myproject.local
   DocumentRoot /var/www/myproject/public

   <Directory /var/www/myproject/public>
      AllowOverride All
      Require all granted
   </Directory>
</VirtualHost>
sudo nano /etc/hosts
```

Add:

```
127.0.0.1 myproject.local
sudo a2ensite myproject.conf
sudo apache2ctl configtest
sudo systemctl reload apache2
```

## Q20. Create a MySQL database user with limited privileges and configure PHP to connect using that user.

```
CREATE DATABASE myproject_db;

CREATE USER 'project_user'@'localhost'
IDENTIFIED BY 'StrongPassword123!';

GRANT SELECT, INSERT, UPDATE, DELETE
ON myproject_db.*
TO 'project_user'@'localhost';

FLUSH PRIVILEGES;
```

## database.php

```
<?php
$host = 'localhost';
$dbname = 'myproject_db';
$username = 'project_user';
$password = 'StrongPassword123!';

try {
    $pdo = new PDO(
        "mysql:host=$host;dbname=$dbname;charset=utf8mb4",
        $username,
        $password
    );

    echo "Database Connected Successfully";
} catch (PDOException $e) {
    echo "Database Connection Failed";
}
?>
```